Aarhus University

Department Of Computer Science

Machine Learning

Group 30

Handin 1

Author	*Student number*
Søren M. Damsgaard	**202309814**
Alexander Eriksen	**202408929**
Thor Behrmann	**202410271**

October 9, 2026



Goofy ah Machines

PART 1: Logistic Regression

Theory

Question 1: Running Time of Mini-Batch Gradient Descent

The running time depends in the number of epochs, since each epoch takes $\mathcal{O}(\backslash)$ time to shuffle and indices the samples, and then it's separated into batches $\frac{n}{B}$ where it takes the gradient and updates the weights, which takes $\mathcal{O}(B \cdot d + d)$ time where (d) is the number of features for each sample, this can simplifies to $\mathcal{O}(B \cdot d)$ and if we sum over all the batches we get a running time $\mathcal{O}(n \cdot d)$ for each epoch, and over all epochs the total running time for mini-batch GD is

$$\mathcal{O}(epochs \cdot n \cdot d)$$

Question 2: Sanity Check (Pixel Permutation)

The placement and ordering of the pixels don't matter since the logistic regression treats each pixel independently as it's own feature with it's own weight, so the inner product is still $w^T x = \sum_i w_i x_i$ which means the shape and order has no affect on the performance.

Question 3: Linearly Separable Data

If the data linearly separable, the weights diverges to infinity ($\|w\| \to \infty$), since if the data is separable it implies that there is some weight w' such that $y_i(w'^T x_i) > 0$ for all i, and if we scale w' with some constant $c > 0$ we get a negative log-likelihood cost

$$\lim_{c\to\infty} E_{in}(cw') = \lim_{c\to\infty} \frac{1}{n} \sum_{i=1}^{n} \ln(1 + e^{-cy_i w'^T x_i}) = 0$$

and gradient descent keeps taking steps that increase $||w|| \to \infty$ without converging on a single weight.

Code

Summary and Results

We trained the Logistic Regression classifier on the Danish CVR industry codes dataset using bag-of-words text features (epochs = 50, batch_size = 16, lr = 0.1):

- **In-sample Accuracy:** 97.54%
- **Test Accuracy:** 95.34% (> 95% target benchmark)

Plot comments: Cost decreases smoothly from $\sim$ 0.30 to $\sim$ 0.08 with stable convergence around epoch 40, verifying that $\eta = 0.1$ is well-suited.

Status: All methods pass unit tests and gradient checks.

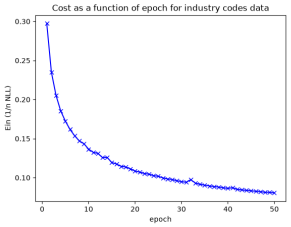


Figure 1: Cost (E_{in}) vs. epoch for industry codes.

Actual Code

```
def cost_grad(self, X, y, w):
    cost = 0
        grad = np.zeros(w.shape)

        ### YOUR CODE HERE
        n = X.shape[0]
        # Margin: s_i = y_i * (x_i . w)
        exponent = y * np.dot(X, w)

        # Average negative log-likelihood
        # numpy has np.logaddexp(0, -s) that computes ln(1 + exp(-s)) with less overflow (
    numerical stability) #trust
        cost = np.mean(np.logaddexp(0, -exponent))

        # Gradient:
        # logistic(-exponent), let arg be negative, to get a positive output
        # transpose X to get correct dimensions
        grad = -np.dot(X.T, y * logistic(-exponent)) / n
        ### END CODE
        assert grad.shape == w.shape
        return cost, grad

if w is None: w = np.zeros(X.shape[1])
        history = []
        ### YOUR CODE HERE
        n = X.shape[0]
        for epoch in range(epochs):
            # Shuffle data before each epoch
            indices = np.random.permutation(n)
            X_shuffled = X[indices]
            y_shuffled = y[indices]

            # Mini-batch gradient descent thing
            for i in range(0, n, batch_size):
                X_batch = X_shuffled[i : i + batch_size]
                y_batch = y_shuffled[i : i + batch_size]
                _, grad = self.cost_grad(X_batch, y_batch, w) # not interested in the cost here
                w = w - lr * grad

            # remember cost for each epoch
            epoch_cost, _ = self.cost_grad(X, y, w) # cost here
            history.append(epoch_cost)
            print(f"Epoch {epoch + 1}/{epochs} - Cost: {epoch_cost:.4f}") #check status after
    each epoch
        ### END CODE
        self.w = w
        self.history = history
```

Listing 1: Logistic Regression `cost_grad` and `fit` methods.

PART 2: Softmax

Theory

Question: Running Time of Softmax Implementation

Let K be the number of classes, n the number of training samples, d the number of features, $B =$ mini-batch size, and epochs the number of training epochs.

Cost and Gradient: For each mini-batch, the algorithm calculates a score for each class and converts the scores into probabilities using the softmax function. It then compares the predicted probabilities with the correct labels to calculate the cost. The gradient is calculated to determine how the weights should be updated to reduce the cost. This takes $\mathcal{O}(B \cdot d \cdot K)$ time per mini-batch. for the entire dataset it takes $\mathcal{O}(n \cdot d \cdot K)$ time

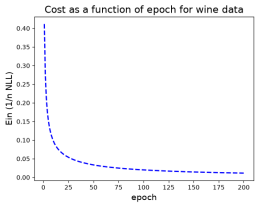
Total running time of Mini-Batch Gradient Descent
During each epoch, the algorithm shuffles the data, processes all mini-batches, updates the weights, and calculates the cost for the entire dataset. This takes $\mathcal{O}(n \cdot d \cdot K)$ time per epoch and the total running time is $\mathcal{O}(epochs \cdot n \cdot d \cdot K)$ time

Code

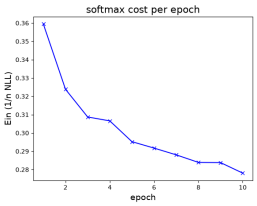
Summary and Results

We evaluated our Softmax Classifier on both the Wine and MNIST digits datasets:

- **Wine Dataset** (200 epochs, $B = 16$, $\eta = 0.1$): In-sample Accuracy: 100.0%, Test Accuracy: 97.78% ($> 90\%$ benchmark).
- **MNIST Digits** (10 epochs, $B = 32$, $\eta = 0.05$): In-sample Accuracy: 92.39%, Test Accuracy: 92.28%.



a Cost per epoch on Wine data.



b Cost per epoch on MNIST digits.

Figure 2: Training cost progression for Softmax regression.

Comments on the plots: Both cost curves show steep initial drops followed by steady flattening without oscillations, reflecting smooth convergence. Numerical stability (log-sum-exp trick) completely prevented overflow during all runs.

Actual Code

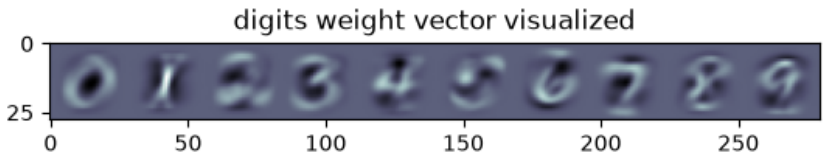


Figure 3: Visualization of learned weight templates for MNIST digits 0–9. Bright values correspond to positive weights matching expected stroke shapes; dark values indicate negative weights for background pixels.

```
def cost_grad(self, X, y, W):
    Yk = one_in_k_encoding(y, self.num_classes)
    n = X.shape[0]
    P = softmax(np.dot(X, W))
    cost = -np.sum(Yk * np.log(P)) / n  # Average NLL cost
    grad = np.dot(X.T, P - Yk) / n      # Gradient of average NLL
    return cost, grad

def fit(self, X, Y, W=None, lr=0.01, epochs=10, batch_size=16):
    if W is None: W = np.zeros((X.shape[1], self.num_classes))
    self.history = []
    n = X.shape[0]
    for epoch in range(epochs):
        perm = np.random.permutation(n)
        for i in range(0, n, batch_size):
            b = perm[i : i + batch_size]
            _, grad = self.cost_grad(X[b], Y[b], W)
            W -= lr * grad
        epoch_cost, _ = self.cost_grad(X, Y, W)
        self.history.append(epoch_cost)
    self.W = W
```

Listing 2: Softmax `cost_grad` and `fit` methods.